inconsistency. For teams practicing BDD, the requirements and executable specifications are the same thing; when the requirements change, the executable specifications are updated directly in a single place. We'll look at this in detail in chapter 9.

DON'T WRITE UNIT TESTS, WRITE LOW-LEVEL SPECIFICATIONS

BDD doesn't stop at the acceptance tests. BDD also helps developers write higher quality code that's more reliable, more maintainable, and better documented.

Developers practicing BDD typically use an *outside-in* approach. When they implement a feature, they start from the acceptance criteria and work down, building whatever is needed to make those acceptance criteria pass. The acceptance criteria define the expected outcomes, and the developer's job is to write the code that produces those outcomes. This is a very efficient, focused way of working. Just as no feature is implemented unless it contributes to an identified business goal, no code is written unless it contributes to making an acceptance test pass, and therefore to implementing a feature.

But it doesn't stop there. Before writing any code, a BDD developer will reason about what this code should actually do and express this in the form of a *low-level executable specification*. The developer won't think in terms of writing unit tests for a particular class, but of writing technical specifications describing how the application should behave, such as how it should respond to certain inputs or what it should do in a given situation. These low-level specifications flow naturally from the high-level acceptance criteria, and help developers design and document the application code in the context of delivering high-level features (see figure 1.9).

For example, the step definition code in figure 1.9 involves creating a new account:

```
@Given("my $accountType account has a balance of $amount")
public void setupInitialAccount(AccountType type, double amount) {
    Account account = Account.ofType(type)
                             .withInitialBalance(amount);
    ...
}
```

Create a new account of given type and with given initial balance.

This leads the developer to write a low-level specification to design the `Account` class. This example uses Spock, a BDD unit testing library for Java and Groovy. The corresponding specification takes the following form:

```
class WhenCreatingANewAccount extends Specification {

    def "account should have a type and an initial balance" () {
        when:
            Account account = Account.ofType(Savings)
                                     .withInitialBalance(100)
        then:
            account.accountType == Savings
            account.balance == 100
    }
}
```